

Reconstrucción Tridimensional de Órganos de Rana

Triangulación de Delaunay (Bowyer–Watson 3D) con Filtrado Alpha-Shape

Laboratorio de Computación Gráfica UNSA-EPCC

Rivas Huanca Diego Raul

Alvarez Astete Jheeremy Manuel

Escuela Profesional de Ciencia de la Computación

Universidad Nacional de San Agustín de Arequipa

27 de julio de 2026

Resumen

Este informe documenta el diseño, implementación y validación de un sistema de reconstrucción tridimensional de órganos de rana a partir de máscaras volumétricas TIFF, utilizando triangulación de Delaunay en 3D (algoritmo de Bowyer–Watson) combinada con un filtrado de tipo alpha-shape para reconstruir superficies no convexas. El sistema incluye un pipeline completo de procesamiento de datos (extracción de superficie, sub-muestreo, cálculo de parámetros de reconstrucción) y una aplicación interactiva de visualización en OpenGL/ImGui. Se valida el algoritmo con casos sintéticos de topología conocida (esfera, cubo, toro), confirmando su correctud mediante la característica de Euler–Poincaré, y se aplica sobre los 17 órganos del conjunto de datos real, documentando limitaciones, tiempos de ejecución y optimizaciones de rendimiento (estructura de datos por hash, paralelización con OpenMP).

Índice

1. Introducción y planteamiento del problema	3
2. Objetivos del proyecto	3
3. Fundamento teórico	4
3.1. Triangulación de Delaunay	4
3.2. Algoritmo de Bowyer–Watson	4
3.3. Circun esfera de un tetraedro	4
3.4. Alpha-shapes	5
3.5. Característica de Euler–Poincaré	5
4. Descripción del sistema desarrollado	5
5. Arquitectura general del proyecto	6
5.1. Módulos principales	6
5.2. Patrón de diseño: Strategy	6
6. Metodología de implementación	7
6.1. Bowyer–Watson en 3D	7
6.2. Extracción de la superficie y alpha-shape	7
6.3. Pipeline de datos	8
7. Proceso de calibración del sistema	8
7.1. Calibración del número de puntos objetivo	8
7.2. Calibración de alpha	9
7.3. Validación cruzada con datos reales	9
8. Reconstrucción tridimensional	9
9. Procesamiento de la nube de puntos	9
10.Resultados obtenidos	10
10.1. Validación con casos sintéticos	10
10.2. Los 17 órganos	10
10.3. Rendimiento	10
10.3.1. Complejidad algorítmica	10
10.3.2. Paralelización con OpenMP	11
11.Validación del sistema y análisis de resultados	12
11.1. Propiedad de Delaunay	12
11.2. Validación topológica mediante Euler–Poincaré	12
11.3. Análisis de resultados sobre datos reales	12
11.4. Limitaciones y dificultades	12
12.Repositorio	13
13.Conclusiones y trabajo futuro	13

1. Introducción y planteamiento del problema

La reconstrucción tridimensional a partir de datos volumétricos segmentados es un problema central en visualización científica y computación gráfica: dado un conjunto de máscaras binarias que indican qué voxeles de un volumen pertenecen a una estructura de interés, se busca generar una representación geométrica (malla de superficie) navegable e interactiva de esa estructura.

En este laboratorio se trabaja sobre un conjunto de datos de 17 máscaras TIFF correspondientes a distintos órganos de una rana (sangre, cerebro, corazón, hígado, músculo, esqueleto, bazo, entre otros), cada una representando un volumen de $136 \times 470 \times 500$ voxeles. El problema a resolver tiene dos componentes:

1. **Geométrico:** dado un conjunto de puntos 3D extraídos de la superficie de un órgano, construir una malla triangulada que reconstruya fielmente su forma – incluyendo concavidades reales del órgano, no solo su envolvente convexa.
2. **De ingeniería de datos:** los volúmenes originales contienen entre 3,935 (bazo) y 4,097,993 (músculo) voxeles activos: es necesario un pipeline que extraiga, reduzca y prepare estos datos a un tamaño manejable para el algoritmo de reconstrucción, sin perder la posición relativa entre los distintos órganos del mismo espécimen.

La dificultad central del componente geométrico es que un cierre convexo (*convex hull*) de la nube de puntos **no** reconstruye correctamente órganos con concavidades – que es el caso general en anatomía. Esto motiva el uso de *alpha-shapes*, una generalización del cierre convexo que permite reconstruir formas no convexas a partir de una triangulación de Delaunay.

2. Objetivos del proyecto

Objetivo general

Implementar un sistema de reconstrucción 3D de órganos de rana a partir de máscaras TIFF segmentadas, utilizando triangulación de Delaunay (Bowyer–Watson en 3D) y visualización interactiva.

Objetivos específicos

- Implementar desde cero el algoritmo de Bowyer–Watson para triangulación de Delaunay en 3 dimensiones (tetraedros).
- Validar la correctud del algoritmo mediante propiedades matemáticas verificables (propiedad de Delaunay, característica de Euler–Poincaré) en casos sintéticos de topología conocida.
- Extender la triangulación con un filtrado alpha-shape para reconstruir superficies no convexas.
- Diseñar un pipeline de procesamiento que extraiga nubes de puntos representativas de la superficie de cada órgano a partir de las máscaras TIFF, preservando la posición relativa entre órganos.

- Implementar una aplicación de visualización interactiva (OpenGL + ImGui) que permita seleccionar, colorear y navegar los 17 órganos reconstruidos.
- Medir y optimizar el rendimiento del sistema (estructuras de datos, paralelización con OpenMP, caché en disco de resultados ya calculados).
- Documentar las limitaciones del enfoque y posibles líneas de trabajo futuro.

3. Fundamento teórico

3.1. Triangulación de Delaunay

Dado un conjunto de puntos $P = \{p_1, \dots, p_n\} \subset \mathbb{R}^3$, una triangulación de Delaunay es una descomposición del casco convexo de P en tetraedros tal que **ningún punto de P se encuentra estrictamente dentro de la circun-esfera de ningún tetraedro** de la triangulación. Esta propiedad (la *propiedad de Delaunay* o *propiedad del círculo/esfera vacía*) maximiza el ángulo diedro mínimo de la triangulación resultante, evitando tetraedros degenerados o extremadamente aplanados.

3.2. Algoritmo de Bowyer–Watson

El algoritmo de Bowyer–Watson [1, 2] construye la triangulación de Delaunay de forma **incremental**: se parte de un tetraedro (o triángulo, en 2D) suficientemente grande como para contener a todos los puntos del conjunto (el *super-tetraedro*), y se insertan los puntos uno a uno. En cada inserción:

1. Se identifican todos los tetraedros cuya circun-esfera contiene al nuevo punto (*tetraedros inválidos*).
2. Se eliminan esos tetraedros, dejando un hueco poliédrico.
3. Se identifican las caras de frontera de ese hueco (caras que pertenecían a un solo tetraedro inválido).
4. Se reconecta el hueco creando un nuevo tetraedro entre cada cara de frontera y el punto insertado.

Al finalizar la inserción de todos los puntos, se descartan los tetraedros que aún comparten algún vértice con el super-tetraedro inicial.

3.3. Circun-esfera de un tetraedro

Dado un tetraedro con vértices $A, B, C, D \in \mathbb{R}^3$, su circun-esfera es la única esfera que pasa por los 4 vértices. El centro O se obtiene resolviendo el sistema lineal 3×3 que surge de imponer $\|O - A\|^2 = \|O - B\|^2 = \|O - C\|^2 = \|O - D\|^2$:

$$\begin{bmatrix} (B - A) \\ (C - A) \\ (D - A) \end{bmatrix} O = \frac{1}{2} \begin{bmatrix} \|B\|^2 - \|A\|^2 \\ \|C\|^2 - \|A\|^2 \\ \|D\|^2 - \|A\|^2 \end{bmatrix} \quad (1)$$

Este sistema se resuelve mediante la regla de Cramer (determinantes 3×3); el radio de la circun-esfera es $r = \|O - A\|$.

3.4. Alpha-shapes

Un *alpha-shape* [3] es una generalización del casco convexo parametrizada por un valor α : dada la triangulación de Delaunay completa de un conjunto de puntos, se descartan los tetraedros cuya circun-esfera tiene radio mayor a α , y se toma la frontera de los tetraedros restantes como superficie reconstruida. Intuitivamente, α controla el tamaño máximo de “hueco” que el algoritmo puede atravesar: valores grandes de α tienden al casco convexo; valores pequeños fragmentan la nube de puntos en componentes desconectadas.

3.5. Característica de Euler–Poincaré

Para una superficie triangulada, cerrada y orientable de género g (número de “agujeros”), se cumple:

$$V - E + F = 2 - 2g \quad (2)$$

donde V , E , F son la cantidad de vértices, aristas y caras. Para una triangulación donde cada arista es compartida por exactamente 2 caras ($2E = 3F$), esto se reduce a:

$$F = 2V - 4(1 - g) \quad (3)$$

Para género 0 (topológicamente una esfera, sin agujeros): $F = 2V - 4$. Para género 1 (un toro, con un agujero): $F = 2V$. Esta relación se usa en este trabajo como **criterio de validación independiente** del algoritmo (Sección 11).

4. Descripción del sistema desarrollado

El sistema se compone de dos partes que se comunican mediante archivos en disco (nubes de puntos `.xyz` y un manifiesto `manifest.tsv`):

1. **Pipeline de datos (Python)**: procesa los TIFF originales, extrae la cáscara de voxels de cada órgano, la sub-muestra a una nube de puntos manejable, calcula estadísticas para sugerir un valor de α , y exporta todo a disco.
2. **Motor de reconstrucción y visualización (C++ / OpenGL)**: implementa el algoritmo de Bowyer–Watson, el filtrado alpha-shape, la construcción de mallas GPU, y una interfaz gráfica interactiva (ImGui) para seleccionar y navegar los órganos reconstruidos.

Se desarrollaron tres ejecutables sobre el mismo núcleo de algoritmo: uno de validación por consola (sin dependencias gráficas), uno de validación visual con formas sintéticas y datos reales cargados a mano, y el visor final del sapo completo (con y sin caché en disco).

5. Arquitectura general del proyecto

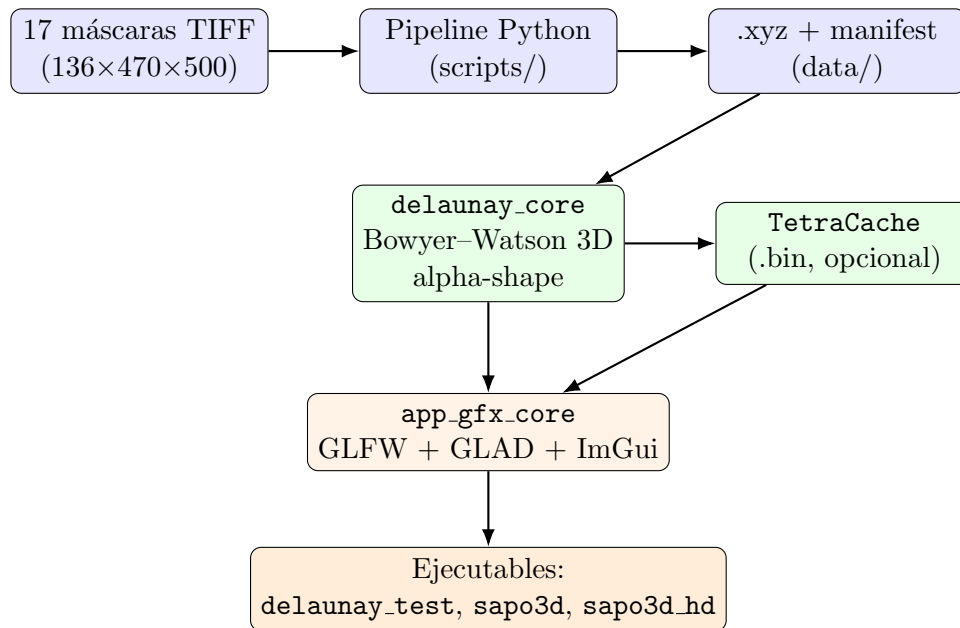


Figura 1: Arquitectura general del sistema.

5.1. Módulos principales

`src/delaunay/` Núcleo del algoritmo, **sin dependencias gráficas**: `Vec3` (vector 3D en `double` para precisión numérica), `Delaunay3D` (Bowyer-Watson + alpha-shape + extracción de superficie), `IDelaunayStrategy` (patrón *Strategy*), `PointCloudIO` / `ManifestIO` (lectores de texto plano), `TetraCache` (caché binario).

`src/core/` Capa gráfica reutilizable: `Application` (ventana GLFW + contexto ImGui + loop principal), `Shader`, `Camera` (cámara orbital), `MeshBuilder` (construcción de buffers GPU).

`apps/` Tres ejecutables: `synthetic_test` (validación visual con formas sintéticas), `sapo3d` (reconstrucción completa), `sapo3d_hd` (ídem, con caché persistente en disco).

`scripts/` Pipeline de datos en Python: `tiff_utils.py` (extracción de cáscara, submuestreo, estadísticas), `manifest_utils.py` (lectura/escritura del manifiesto), `extract_surface.py` / `extract_all.py` (extracción de uno o todos los órganos), `report_voxel_counts.py` (reporte de referencia).

5.2. Patrón de diseño: Strategy

La interfaz `IDelaunayStrategy` desacopla el resto del sistema de la implementación concreta del algoritmo de triangulación, permitiendo sin modificar el código cliente – intercambiar la estrategia de reconstrucción (por ejemplo, para comparar variantes del algoritmo o futuras optimizaciones):

```
1 class IDelaunayStrategy {
2 public:
```

```

3   virtual ~IDelaunayStrategy() = default;
4   virtual std::vector<Tetrahedron> triangulate(
5       const std::vector<Vec3>& points) = 0;
6   virtual const char* name() const = 0;
7 };
8
9 class BowyerWatsonStrategy : public IDelaunayStrategy {
10 public:
11     std::vector<Tetrahedron> triangulate(
12         const std::vector<Vec3>& points) override {
13         return BowyerWatson3D::triangulate(points);
14     }
15     const char* name() const override { return "Bowyer-Watson"; }
16 };

```

Listing 1: Patrón Strategy sobre el algoritmo de triangulación

6. Metodología de implementación

6.1. Bowyer–Watson en 3D

El Algoritmo 1 resume la implementación central. La estructura de datos principal es un **Tetrahedron** con 4 índices hacia el arreglo de puntos, más el centro y radio al cuadrado de su circunfera (cacheados para no recalcularlos en cada test de punto).

Algorithm 1 Bowyer–Watson 3D

```

1: procedure TRIANGULATE(puntos)
2:   superTet ← CREAMSUPER-TETRAEDRO(puntos)
3:   tets ← {superTet}
4:   for cada punto p en puntos do
5:     malos ← {t ∈ tets : p dentro de circunfera(t)}
6:     caras ← todas las caras de los tetraedros en malos
7:     frontera ← caras que aparecen una sola vez en caras
8:     tets ← (tets \ malos)
9:     for cada cara f en frontera do
10:      tets ← tets ∪ {nuevoTetraedro(f, p)}
11:   end for
12: end for
13: return {t ∈ tets : t no comparte vértice con superTet}
14: end procedure

```

El super-tetraedro se construye tomando el centro y la diagonal del *bounding box* de los puntos de entrada, y generando 4 vértices en configuración de tetraedro regular (vértices alternados de un cubo) escalados a 5 veces esa diagonal suficientemente grande como para garantizar que ningún punto de entrada quede fuera de él.

6.2. Extracción de la superficie y alpha-shape

```

1 std::vector<Tetrahedron> BowyerWatson3D::filterByAlpha(
2     const std::vector<Tetrahedron>& tets, double alphaRadius)
3     {
4     double alphaSq = alphaRadius * alphaRadius;
5     std::vector<Tetrahedron> result;
6     for (const auto& t : tets) {
7         if (t.radiusSq <= alphaSq) result.push_back(t);
8     }
9     return result;
10 }

```

Listing 2: Filtrado alpha-shape sobre la triangulación completa

La extracción de caras de frontera (`boundaryFaces`) identifica, entre todas las caras de los tetraedros restantes, las que pertenecen a un único tetraedro (las caras internas, compartidas por dos, se descartan). Este paso se optimizó de $O(F^2)$ a $O(F)$ amortizado mediante una tabla hash – ver Sección 10.3.

6.3. Pipeline de datos

1. **Extracción de cáscara:** de la máscara binaria sólida, se aplica erosión binaria y se resta del original, quedándose solo con los voxeles de borde. Reduce el conteo de puntos entre $5\times$ y $20\times$ según el órgano antes de cualquier otro procesamiento.
2. **Sub-muestreo por grilla 3D:** se particiona el espacio en celdas cúbicas y se toma un punto representante por celda ocupada, ajustando el tamaño de celda por búsqueda binaria hasta acercarse al conteo de puntos objetivo. Da una cobertura espacial más uniforme que un muestreo puramente aleatorio.
3. **Centrado compartido:** todos los órganos se centran respecto al centro del volumen completo del TIFF (idéntico para los 17 archivos, que comparten *shape*), **no** respecto al centroide de cada órgano individual – esto preserva la posición relativa entre órganos al combinarlos en la reconstrucción final.
4. **Estadística de vecino más cercano:** se calcula la distancia mediana al vecino más cercano sobre la nube ya sub-muestreada, y se sugiere $\alpha \in [2,0, 3,0]\times$ esa mediana como punto de partida.

7. Proceso de calibración del sistema

7.1. Calibración del número de puntos objetivo

Dado que el tamaño de la cáscara de voxeles varía en más de dos órdenes de magnitud entre órganos (1,385 para el bazo hasta 822,268 para el músculo), se calibró una función de escalado $\text{target} = \text{clip}(K\sqrt{\text{cáscara}}, 300, 3000)$, con $K = 16$ ajustado empíricamente a partir del caso del bazo ($\text{cáscara} = 1385 \rightarrow 600$ puntos, resultado visualmente aceptable en las primeras pruebas).

7.2. Calibración de alpha

El valor de α se calibró primero de forma **manual y exhaustiva** sobre el caso sintético del toro (Sección 11), donde el valor óptimo se pudo verificar de forma exacta contra la predicción de Euler–Poincaré ($\alpha = 2,5$, sobre una nube con distancia entre puntos vecinos $\approx 1,0$, es decir $\alpha \approx 2,5 \times$ esa distancia). Ese factor ($2,0\text{--}3,0 \times$ la distancia mediana al vecino más cercano) se generalizó como heurística automática para los 17 órganos reales, evitando así tener que ajustar el parámetro a mano para cada uno.

7.3. Validación cruzada con datos reales

Se aplicó la heurística sobre el bazo (574 puntos, $\alpha_{auto} \approx 2,5$) y el hígado (3039 puntos, $\alpha_{auto} \approx 9,6$), confirmando visualmente formas reconocibles en ambos casos, con huecos menores en zonas de baja densidad de puntos (extremidades finas) – documentado como limitación en la Sección 11.4.

8. Reconstrucción tridimensional

Una vez obtenidos los tetraedros filtrados por alpha-shape, la reconstrucción de la malla de superficie sigue estos pasos:

1. `boundaryFaces()` extrae las caras de frontera del conjunto de tetraedros filtrado.
2. Por cada cara triangular, se calcula su normal (mediante producto cruz de dos de sus aristas) para *flat shading*.
3. Se construye un buffer de vértices intercalado (posición + normal, 6 floats por vértice, 3 vértices por cara, sin *index buffer*) y se sube a la GPU (`MeshBuilder::buildSurfaceMesh`).
4. El render usa un modelo de iluminación de Phong simplificado (ambiente + difusa + especular) en GLSL, con una cámara orbital controlada por arrastre de mouse y scroll.

Para depuración también se construye una malla de líneas con las aristas únicas de los tetraedros (sin filtrar por cara de frontera), útil para inspeccionar visualmente la estructura interna de la triangulación.

9. Procesamiento de la nube de puntos

El procesamiento de la nube de puntos ocurre enteramente en el lado Python del pipeline (Sección 6, pipeline de datos), antes de que los datos lleguen al motor de triangulación en C++. Los archivos intermedios (`.xyz`, texto plano con una fila "`x y z`" por punto) se versionan de forma independiente de los TIFF originales, lo que permite regenerar una nube de puntos con otro nivel de detalle sin volver a tocar el algoritmo de reconstrucción.

El sistema de *caché binario* (`TetraCache`) opera un nivel más abajo: una vez que una nube de puntos fue triangulada, el resultado completo (puntos + tetraedros, con su circunsfera ya calculada) se persiste en disco (`data/cache/<órgano>_<N puntos>.bin`), evitando re-ejecutar Bowyer–Watson en corridas posteriores del programa. El nombre del archivo de caché incluye la cantidad de puntos, por lo que cambiar el nivel de detalle de un órgano no requiere invalidar manualmente ningún caché existente.

10. Resultados obtenidos

10.1. Validación con casos sintéticos

Cuadro 1: Resultados de validación con formas de topología conocida.

Caso	Puntos	Tetraedros	Caras de frontera	Violaciones D
Esfera (genus 0)	40	123	76 (esperado 76, $2V-4$)	0
Cubo aleatorio	60	285	46 (envolvente convexa)	0
Toro sin α (genus 1)	288	1524	332 (casco convexo, tapa el agujero)	0
Toro con $\alpha = 2,5$	288	1128	576 (esperado 576, $2V$)	0

10.2. Los 17 órganos

Cuadro 2: Conteo de voxeles originales y puntos finales usados por órgano.

Órgano	Voxeles activos	Voxeles cáscara	Puntos usados	α_{auto}
blood	35,492	26,035	2,492	5.00
brain	19,172	5,374	1,200	3.54
duodenum	38,224	12,182	1,741	3.54
eye	53,231	9,024	1,525	3.54
eyeRetna	37,146	13,317	1,832	3.54
eyeWhite	16,085	5,165	1,158	3.54
heart	37,196	8,648	1,561	3.54
ileum	27,916	9,874	1,516	3.54
kidney	39,245	12,199	1,753	3.54
Intestine	51,660	9,634	1,625	3.54
liver	205,245	42,951	3,039	5.00
lung	47,454	13,210	1,782	3.54
muscle	4,097,993	822,268	2,933	17.32
nerve	24,314	16,649	1,987	3.54
skeleton	440,640	198,340	3,015	8.29
spleen	3,935	1,385	574	2.50
stomach	134,323	67,602	2,975	5.59
Total	5,309,271	1,273,857	32,708	—

10.3. Rendimiento

10.3.1. Complejidad algorítmica

El algoritmo de Bowyer–Watson, tal como está implementado (sin estructuras aceleradoras de tipo *walk* o *kd-tree*), tiene complejidad $O(n^2)$ en el peor caso: por cada uno de los n puntos insertados, se recorre el conjunto de tetraedros existentes (que crece proporcionalmente a n) para clasificarlos.

La extracción de caras de frontera (`boundaryFaces`) se implementó inicialmente comparando cada cara contra todas las demás ($O(F^2)$, con $F = 4 \times |\text{tetraedros}|$), y se optimizó a $O(F)$ amortizado usando una tabla hash sobre los 3 índices (ordenados canónicamente) de cada cara:

Cuadro 3: Impacto de la optimización de `boundaryFaces()` (3000 puntos, 19,511 tetraedros, 78,044 caras).

Implementación	Tiempo
Comparación por pares, $O(F^2)$	2.20 s
Tabla hash, $O(F)$ amortizado	0.005 s

10.3.2. Paralelización con OpenMP

Dentro de cada inserción de Bowyer–Watson, la clasificación de tetraedros (¿el punto cae dentro de su circunferencia?) es independiente entre tetraedros, por lo que se paralelizó con `#pragma omp parallel for` (activado solo por encima de 500 tetraedros, para no pagar el overhead de creación de hilos en nubes chicas):

```

1 std::vector<char> isBad(tets.size(), 0);
2 #pragma omp parallel for schedule(static) if (numTets > 500)
3 for (long ti = 0; ti < numTets; ++ti) {
4     if (inCircumsphere(tets[ti], p, allPoints, epsilonRel))
5         isBad[ti] = 1;
6 }

```

Listing 3: Clasificación de tetraedros paralelizada con OpenMP

Cuadro 4: Tiempos de triangulación medidos en la máquina de desarrollo (16 núcleos lógicos, build Release), forzando 1 núcleo vs. usando los 16 disponibles.

Puntos	Tetraedros	1 núcleo	16 núcleos	Razón (16n / 1n)
500	3,002	0.0307 s	0.0597 s	1.94× más lento
1500	9,647	0.1027 s	0.2055 s	2.00× más lento
3000	19,511	0.3765 s	0.5503 s	1.46× más lento
5000	32,735	1.4733 s	1.7729 s	1.20× más lento

Hallazgo: contrario a la hipótesis inicial, paralelizar con `#pragma omp parallel for` resultó *consistentemente más lento* que la versión secuencial, en las 4 nubes de puntos evaluadas sobre hardware real de 16 núcleos lógicos. La causa raíz es de granularidad: la región paralela se abre y cierra **una vez por cada punto insertado** (miles de veces por corrida), y el trabajo real dentro de cada apertura (comparar un punto contra los tetraedros vigentes) es demasiado pequeño para amortizar el costo de coordinar 16 hilos en cada una. El overhead de sincronización domina sobre la ganancia de paralelismo.

Un dato a favor de la hipótesis original: la desventaja se reduce a medida que crece la nube de puntos (de 2,00× más lento en 1500 puntos a 1,20× en 5000), consistente con un overhead aproximadamente fijo por llamada que se diluye a medida que aumenta el trabajo real por llamada. Esto sugiere que, en nubes bastante más grandes que las usadas en este proyecto, la paralelización *podría* llegar a compensar, pero confirmar esto exigiría una reestructuración distinta (mantener un único equipo de hilos persistente para toda la llamada a `triangulate()`, usando `#pragma omp parallel` una sola vez con secciones `single/for` internas, en vez de abrir una región nueva por punto), que no se implementó por alcance y por no poder validarse con certeza en el entorno de desarrollo. Se documenta

como limitación medida y como línea de trabajo futuro concreta (Sección 11.4). El código actual con OpenMP es **funcionalmente correcto** (produce resultados idénticos a la versión secuencial en todos los casos de validación), solo no resultó beneficioso con esta granularidad.

Se evaluó el uso de CUDA para acelerar aún más la triangulación, y se descartó: Bowyer–Watson incremental es inherentemente secuencial *entre* inserciones (el estado de la malla tras insertar el punto i es precondition para insertar el punto $i + 1$), por lo que no se presta a paralelización masiva en GPU sin reemplazar el algoritmo completo por un enfoque distinto (p. ej. *divide and conquer* paralelo). Además, la GPU del sistema ya se emplea activamente para el render (shaders GLSL, buffers de vértices), que es la etapa que efectivamente se beneficia de paralelismo masivo en este proyecto.

11. Validación del sistema y análisis de resultados

11.1. Propiedad de Delaunay

Para cada tetraedro resultante, se verificó que ningún punto de la nube de entrada (fuera de sus 4 vértices) cae estrictamente dentro de su circunferencia. En los 4 casos sintéticos evaluados (esfera, cubo, toro con y sin α), el conteo de violaciones fue **0** en todos los casos.

11.2. Validación topológica mediante Euler–Poincaré

El caso del toro es la prueba de validación más informativa del proyecto: al ser una superficie de género 1 (con un agujero real), la característica de Euler–Poincaré predice $F = 2V$ caras en una triangulación cerrada correcta. Con $V = 288$ puntos, eso son 576 caras esperadas. El filtrado con $\alpha = 2,5$ produjo **exactamente 576** caras de frontera – una coincidencia exacta que confirma que el alpha-shape reconstruye la *topología real* del objeto (agujero incluido), y no solamente su casco convexo (que en el mismo caso dio 332 caras, un valor topológicamente incorrecto para un toro).

11.3. Análisis de resultados sobre datos reales

La aplicación sobre los 17 órganos reales confirma que el enfoque escala razonablemente entre estructuras pequeñas y compactas (bazo) y grandes/complejas (hígado, con lóbulos), produciendo reconstrucciones reconocibles en ambos extremos. Sin embargo, se identificaron limitaciones que se detallan a continuación.

11.4. Limitaciones y dificultades

- **Alpha-shape de radio único por órgano:** un solo valor de α no siempre reconstruye bien zonas anchas y finas del mismo órgano simultáneamente. En el bazo y el hígado aparecen huecos menores en extremidades/protuberancias finas, donde la distancia entre puntos vecinos crece localmente y el mismo α que conecta bien el cuerpo principal ya no alcanza ahí. Una alpha-shape *adaptativa* (radio variable según densidad local) resolvería esto, pero excede el alcance de este laboratorio.

- **Sub-muestreo con tope fijo de puntos:** *muscle* (822K voxeles de cáscara) y *skeleton* (198K) se comprimen al mismo techo de ~ 3000 puntos que órganos mucho más chicos como el bazo (1385) – esto implica menos detalle relativo en las estructuras más grandes y complejas del espécimen.
- **Complejidad algorítmica sin aceleración espacial:** la implementación usa búsqueda lineal sobre los tetraedros existentes en cada inserción; es adecuada para las nubes de algunos miles de puntos usadas aquí, pero no escalaría bien a nubes de cientos de miles de puntos sin una estructura de aceleración espacial (p. ej. *walk* guiado por localidad, o un *kd-tree*).
- **Supuesto de voxel isotrópico:** se asume que todas las máscaras comparten el mismo *shape* de volumen y que los voxeles son isotrópicos (mismo espaciado en los 3 ejes) – válido para este conjunto de datos puntual, pero no es un supuesto general para cualquier dataset médico (p. ej. tomografías con distinto espaciado entre slices y dentro de cada slice).

12. Repositorio

Link del código: <https://github.com/DiegoRivas1/cg-delaunay-sapo3d.git>

13. Conclusiones y trabajo futuro

Conclusiones

- Se implementó y validó exitosamente el algoritmo de Bowyer–Watson para triangulación de Delaunay en 3 dimensiones, confirmando su correctud mediante dos criterios independientes: la propiedad de Delaunay (0 violaciones en todos los casos) y la característica de Euler–Poincaré (coincidencia exacta en el caso del toro, $F = 576 = 2V$).
- El filtrado alpha-shape resultó necesario y efectivo para reconstruir formas no convexas – sin él, el sistema reconstruiría únicamente el casco convexo de cada órgano, perdiendo toda concavidad anatómica real.
- El pipeline de datos (extracción de cáscara + sub-muestreo por grilla + centrado compartido) permitió procesar los 17 órganos de forma automática y consistente, preservando su posición relativa dentro del espécimen.
- Las optimizaciones de rendimiento efectivamente exitosas (tabla hash para **boundaryFaces**: de 2.20 s a 0.005 s; caché binario en disco para no re-triangular entre corridas) redujeron significativamente el tiempo de uso interactivo del sistema, sin alterar la correctud de los resultados (verificado comparando resultados antes/después de cada optimización). La paralelización con OpenMP, en cambio, se midió rigurosamente (build Release, 1 vs. 16 núcleos, 4 tamaños de nube) y resultó **consistentemente más lenta** que la versión secuencial con la granularidad actual – un hallazgo negativo real, cuya causa raíz (demasiadas regiones paralelas pequeñas) se identificó y documentó junto con la reestructuración correcta pendiente (Sección 10.3).

Trabajo futuro

- Alpha-shape adaptativa (radio variable por región, según densidad local de puntos) para reducir los huecos en extremidades finas.
- Estructura de aceleración espacial (kd-tree o *walk*) para bajar la complejidad práctica de la inserción de puntos, permitiendo subir el tope de puntos por órgano sin degradar el tiempo de respuesta interactivo.
- Reestructurar la paralelización con OpenMP para usar un único equipo de hilos persistente durante toda la llamada a `triangulate()` (una región `#pragma omp parallel` con secciones `single/for` internas), en vez de abrir una región nueva por cada punto insertado – la medición actual (Sección 10.3) muestra que la version actual es consistentemente más lenta que la secuencial, con el overhead diluyéndose a medida que crece la nube de puntos, lo que sugiere que una implementación con menos overhead de sincronización sí podría aportar una mejora real.
- Suavizado de malla (p. ej. Laplacian smoothing) como posproceso, para reducir el aspecto “faceteado” de las superficies reconstruidas con pocos puntos.
- Exploración de un enfoque de Delaunay paralelo por *divide and conquer* si se buscara aprovechar GPU/CUDA para la etapa de triangulación (no solo para el render).
- Extensión del pipeline de datos para soportar voxeles anisotrópicos (espaciado distinto por eje), generalizando su uso a otros conjuntos de datos volumétricos médicos.

Referencias

- [1] Bowyer, A. (1981). *Computing Dirichlet Tessellations*. The Computer Journal.
- [2] Watson, D. F. (1981). *Computing the n-dimensional Delaunay Tessellation with Application to Voronoi Polytopes*. The Computer Journal.
- [3] Edelsbrunner, H., & Mücke, E. P. (1994). *Three-Dimensional Alpha Shapes*. ACM Transactions on Graphics.